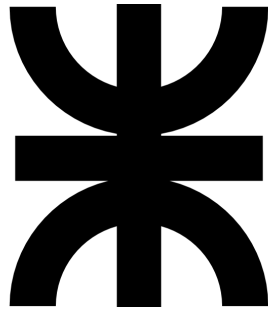


UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Córdoba



Licenciatura en Automatización y Control

Visión Artificial

Actividad Práctica No. 2

Docentes: Ing. González, Hernando Alexis
Ing. Miklosa, Pablo

2025

1 Procedimiento

1.1 Objetivo general

Desarrollar un sistema de visión artificial capaz de detectar los colores primarios (rojo, verde y amarillo) en tiempo real mediante una cámara web, y transmitir esta información por puerto serial a un microcontrolador **Arduino o similar** que realizará acciones específicas según el color detectado.

1.2 Prerrequisitos

- Conocimientos básicos de **Python** y **OpenCV**.
- Familiaridad con microcontroladores **Arduino**.
- Conceptos fundamentales del espacio de color **HSV**.
- Elementos necesarios: Cámara web, placa Arduino, *LEDs* (rojo, verde, amarillo), resistencias y cables.

1.3 Parte 1: Análisis del problema

1. Análisis preliminar

- (a) Investigar y explicar las diferencias entre los espacios de color **RGB** y **HSV**, justificando por qué **HSV** es más adecuado para segmentación por color.
- (b) Documentar los rangos *HSV* típicos para los colores rojo, verde y amarillo, contemplando variaciones de iluminación.

2. Diseño del sistema

Elaborar un diagrama de flujo que incluya:

- Captura de imagen.

- Procesamiento y detección de colores.
- Toma de decisiones.
- Comunicación serial.
- Actuación del microcontrolador.

1.4 Parte 2: Configuración del entorno

1. Instalación de bibliotecas

```
pip install numpy opencv-python pyserial
```

Explicar brevemente la función de cada biblioteca en el proyecto.

2. Configuración inicial

Implementar un script que capture y muestre video en tiempo real:

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    cv2.imshow('Original', frame)

    if cv2.waitKey(1) & 0xFF == ord('s'):
        break

cap.release()

cv2.destroyAllWindows()
```

1.5 Parte 3: Detección de colores

1. Convertir cada *frame* al espacio **HSV** y visualizarlo.

2. Definir rangos **HSV** para los colores (rojo tiene dos rangos).
3. Crear máscaras binarias con `cv2.inRange()` y combinarlas para rojo con `cv2.add()`.
4. Mostrar:
 - Máscaras binarias.
 - Resultado de `cv2.bitwise_and()` por color.
5. Cuantificar píxeles por color con `cv2.countNonZero()` y establecer un umbral mínimo.

1.6 Parte 4: Comunicación serial

1. Investigar **pyserial** e inicializar la comunicación con Arduino.
2. Desarrollar un algoritmo que:
 - Determine el color predominante.
 - Reduzca ruido: contador de *frames*, comparación con último color enviado y umbral de píxeles.
3. Enviar caracteres por color: R, G, Y.
4. Manejar excepciones y liberar recursos correctamente (opcional).

1.7 Parte 5: Programación del Arduino

1. Diseñar circuito: conectar *LEDs* (rojo, verde, amarillo) a pines.
2. Configurar Arduino:
 - Definición de pines y puerto serial.
 - Inicialización de pines como salidas.
3. Leer puerto serial continuamente (**switch-case**) y encender/apagar LEDs.
4. Proveer retroalimentación a través del monitor serial.

1.8 Parte 6: Integración y pruebas

1. Probar con objetos de distintos colores y ajustar rangos *HSV*.
2. Insertar mensajes de depuración en Python y Arduino.
3. Documentar código y elaborar informe técnico con:
 - Algoritmos implementados.
 - Diagramas de flujo.
 - Análisis de resultados y limitaciones.
 - Propuestas de mejora.

1.9 Parte 7: Desafíos adicionales (opcional)

1. **Mejora de robustez**
Filtrado, morfología y rangos adaptativos *HSV*.
2. **Expansión de funcionalidades**
Detección de formas y control de robot móvil.
3. **Interfaz gráfica**
Ajuste de parámetros en tiempo real e histograma *HSV*.

1.10 Criterios de evaluación

- **Funcionamiento técnico** (40 %): detección, comunicación y respuesta del Arduino.
- **Calidad de la implementación** (30 %): estructura, manejo de errores y optimización.
- **Documentación y análisis** (20 %): claridad, análisis crítico y mejoras.
- **Creatividad y valor agregado** (10 %): soluciones innovadoras y extras relevantes.

1.11 Fecha de entrega

Formato de entrega:

- Código fuente comentado (Python y Arduino).
- Informe técnico en PDF.
- Video demostrativo (máx. 5 min).

Fecha límite: *12/06/2025*.

¡Buena suerte con el desarrollo!